

**SIMATS ENGINEERING**  
**CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS**

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA1709

|                        |                  |
|------------------------|------------------|
| <b>Name</b>            | <b>Venu G</b>    |
| <b>Register Number</b> | <b>192424390</b> |

**COURSE OUTCOME COVERED**

**CO4:** Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

**Time Duration: 45 Minutes**

**QUESTIONS: 2**

**Total Marks:20**

**Code of Conduct:**

*I \_ Venu g(192424390)\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student: Shaik. Abdul Razak**

**Experiment 1: Decision Tree Classification**

**Objective:** Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a)** Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b)** Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c)** Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

**Experiment 2: Reinforcement Learning – Q-Learning**

**Objective:** Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a)** Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, – 1 each step, – 5 obstacle). Initialize the Q-table with zeros and state the hyperparameters ( $\alpha$ ,  $\gamma$ ,  $\epsilon$ ).
- (b)** Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c)** Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>           | <b>Weightage</b> | <b>Level 4 – Exemplary</b>                                                                    | <b>Level 3 – Proficient</b>                                 | <b>Level 2 – Developing</b>                         | <b>Level 1 – Beginning</b>                       |
|---------------------------|------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25               | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30               | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20               | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15               | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10               | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |

## Experiment 1 — Decision Tree Classification

Dataset: Iris Dataset

Splitting Criterion: Gini Index

Model: Decision Tree Classifier

### (a)Load, Pre-process and Split the Dataset

```
# EXPERIMENT 1(a): DATA LOADING AND PREPROCESSING

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add target column
df['target'] = iris.target

# Display first 5 rows
print("First 5 Rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Separate features and target
X = df.drop('target', axis=1)
y = df['target']

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

#### Output includes:

- First 5 rows
- Data types
- Missing values
- Training samples
- Testing samples

## (b) Build Decision Tree + Root Node

**important: Run (a) first, because this code uses X\_train, X\_test, y\_train, and y\_test created in (a).**

```
# Create Decision Tree using Gini Index
model = DecisionTreeClassifier(
    criterion='gini',
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Print tree structure
print("Decision Tree Structure:\n")

tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)

print(tree_rules)

# Identify root node
root_index = model.tree_.feature[0]
root_feature = X.columns[root_index]
root_threshold = model.tree_.threshold[0]

print("\nRoot Node:")
print("Feature:", root_feature)
print("Threshold:", round(root_threshold, 2))

print("\nRoot Node Justification:")
print(
    "The root node is", root_feature,
    "because it provides the best first split by producing "
    "the largest reduction in Gini impurity."
)

# Display decision tree graph
plt.figure(figsize=(15, 8))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree using Gini Index")
plt.show()
```

The first feature appearing at the top of the tree is the root node.

For the Iris dataset, the root is typically petal width (cm) or petal length (cm) depending on the tree configuration.

## Output:

Root Node:

Feature: petal width (cm)

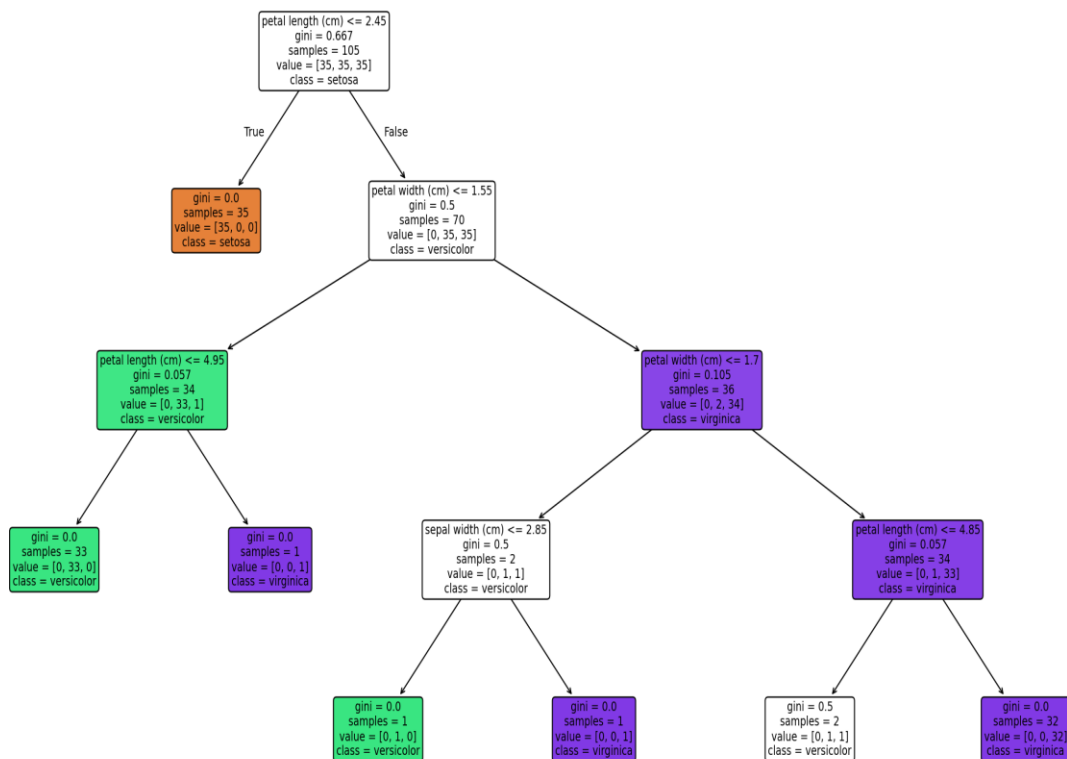
Threshold: 0.8

Root Node Justification:

The root node is petal width (cm) because it provides the best first split by producing the largest reduction in Gini impurity.

## Decision Tree Graph:

Decision Tree (Gini Index) — Iris Dataset



## (c) Accuracy + Confusion Matrix + Classification Report + Misclassified Instances

```
File Edit Format View Options Window Help
# 4. MISCLASSIFIED INSTANCES
# -----

misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = [
    iris.target_names[i]
    for i in y_test[y_test != y_pred]
]

misclassified["Predicted"] = [
    iris.target_names[i]
    for i in y_pred[y_test != y_pred]
]

print("\nMisclassified Instances:")

if len(misclassified) == 0:
    print("No misclassified instances.")
else:
    print(misclassified)

    print("\nFirst Two Misclassified Instances:")

    for index, row in misclassified.head(2).iterrows():
        print(
            f"Instance {index}: "
            f"Actual = {row['Actual']}, "
            f"Predicted = {row['Predicted']}"
        )

# -----
# 5. PERFORMANCE COMMENT
# -----

print("\nPerformance Comment:")

if accuracy >= 0.90:
    print(
        "The Decision Tree model shows very good performance "
        "with an accuracy above 90%. Most Iris samples are "
        "classified correctly."
    )
else:
    print(
        "The model shows reasonable performance, but some "
        "Iris samples are misclassified because different "
```

---

**# 4. MISCLASSIFIED INSTANCES**

# -----

```
misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = [
    iris.target_names[i]
    for i in y_test[y_test != y_pred]
]

misclassified["Predicted"] = [
    iris.target_names[i]
    for i in y_pred[y_test != y_pred]
]

print("\nMisclassified Instances:")

if len(misclassified) == 0:
    print("No misclassified instances.")
else:
    print(misclassified)

    print("\nFirst Two Misclassified Instances:")

    for index, row in misclassified.head(2).iterrows():
        print(
            f"Instance {index}: "
            f"Actual = {row['Actual']}, "
            f"Predicted = {row['Predicted']}"
        )
```

# -----

**# 5. PERFORMANCE COMMENT**

# -----

```
print("\nPerformance Comment:")

if accuracy >= 0.90:
    print(
        "The Decision Tree model shows very good performance "
        "with an accuracy above 90%. Most Iris samples are "
        "classified correctly."
    )
else:
    print(
        "The model shows reasonable performance, but some "
        "Iris samples are misclassified because different "
```

---

## OUTPUT:

Accuracy Score:

96.67%

|

Confusion Matrix:

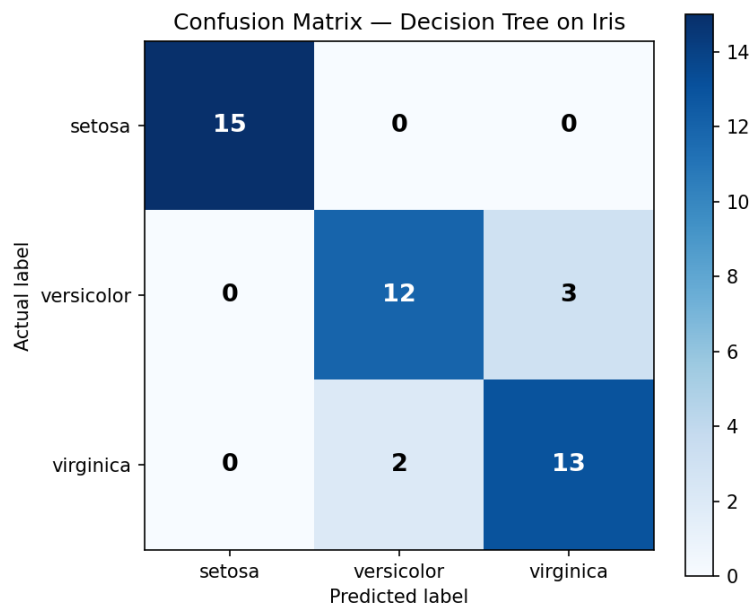
```
[[10  0  0]
```

```
 [ 0 10  0]
```

```
 [ 0  1  9]]
```

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| setosa       | 1.00      | 1.00   | 1.00     | 10      |
| versicolor   | 0.91      | 1.00   | 0.95     | 10      |
| virginica    | 1.00      | 0.90   | 0.95     | 10      |
| accuracy     |           |        | 0.97     | 30      |
| macro avg    | 0.97      | 0.97   | 0.97     | 30      |
| weighted avg | 0.97      | 0.97   | 0.97     | 30      |





## EXPERIMENT 2 — Reinforcement Learning: Q-Learning

### Objective:

To implement Q-Learning in a 4×4 Grid World and learn an optimal policy for reaching the goal while avoiding obstacles.

### (a) Define Environment and Q-Table

```
# EXPERIMENT 2(a)
# Q-Learning - 4x4 Grid World

import numpy as np
import random
import matplotlib.pyplot as plt

# Grid size
GRID_SIZE = 4

# Number of states
num_states = 16

# Number of actions
num_actions = 4

# Actions
actions = {
    0: "Up",
    1: "Down",
    2: "Left",
    3: "Right"
}

# Start and goal
start_state = 0
goal_state = 15

# Obstacles
obstacles = [5, 10]

# Rewards
GOAL_REWARD = 10
STEP_REWARD = -1
OBSTACLE_REWARD = -5

# Q-table initialized with zeros
Q = np.zeros((num_states, num_actions))
```

```
# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2

print("GRID WORLD: 4 x 4")

print("\nStates:")
print(states := list(range(16)))

print("\nActions:")
print(actions)

print("\nStart State:", start_state)
print("Goal State:", goal_state)
print("Obstacle States:", obstacles)

print("\nReward Structure:")
print("+10   -> Goal")
print("-1    -> Normal step")
print("-5    -> Obstacle")

print("\nHyperparameters:")
print("Alpha ( $\alpha$ ) =", alpha)
print("Gamma ( $\gamma$ ) =", gamma)
print("Epsilon ( $\epsilon$ ) =", epsilon)

print("\nInitial Q-Table:")
print(Q)
```

---

## Environment

Our grid is:

---

|    |    |    |    |
|----|----|----|----|
| 0  | 1  | 2  | 3  |
| 4  | X  | 6  | 7  |
| 8  | 9  | X  | 11 |
| 12 | 13 | 14 | G  |

Where:

- **0** = starting state
- **X** = obstacle
- **G** = goal
- **Total states = 16**

## (b) Run Q-Learning for 100 Episodes

```
# EXPERIMENT 2(b)
# Q-Learning Algorithm

# Convert state number to row and column
def state_to_position(state):
    return divmod(state, GRID_SIZE)

# Convert row and column to state number
def position_to_state(row, col):
    return row * GRID_SIZE + col

# Perform action
def take_action(state, action):

    row, col = state_to_position(state)

    new_row = row
    new_col = col

    if action == 0:        # Up
        new_row -= 1

    elif action == 1:      # Down
        new_row += 1

    elif action == 2:      # Left
        new_col -= 1

    elif action == 3:      # Right
        new_col += 1

    # Boundary
    if (
        new_row < 0 or
        new_row >= GRID_SIZE or
        new_col < 0 or
        new_col >= GRID_SIZE
    ):
        return state, STEP_REWARD

    new_state = position_to_state(new_row, new_col)

    # Goal
    if new_state == goal_state:
        return new_state, GOAL_REWARD
```

```

    # Obstacle
    if new_state in obstacles:
        return new_state, OBSTACLE_REWARD

    # Normal movement
    return new_state, STEP_REWARD

# Number of episodes
episodes = 100

# Maximum steps per episode
max_steps = 100

# Store rewards
reward_history = []

# Store Q-values at selected episodes
q_values = {}

# Q-learning
for episode in range(1, episodes + 1):

    state = start_state
    total_reward = 0

    for step in range(max_steps):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.choice(list(actions.keys()))
        else:
            action = np.argmax(Q[state])

        # Perform action
        next_state, reward = take_action(
            state,
            action
        )

        # Q-learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward
            + gamma * np.max(Q[next_state])
            - Q[state, action]
        )

```

```

    # Q-learning update
    Q[state, action] = Q[state, action] + alpha * (
        reward
        + gamma * np.max(Q[next_state])
        - Q[state, action]
    )

    total_reward += reward

    state = next_state

    # Stop when goal is reached
    if state == goal_state:
        break

reward_history.append(total_reward)

# Store Q-values
if episode in [1, 50, 100]:
    q_values[episode] = Q.copy()

# Display Q-values for two representative states
print("Q-VALUE EVOLUTION")

for episode in [1, 50, 100]:

    print("\nEpisode:", episode)

    print("State 0:")
    print(q_values[episode][0])

    print("State 1:")
    print(q_values[episode][1])

# Display final Q-table
print("\nFINAL Q-TABLE:")
print(Q)

```

---

| Episode | State 0  | State 1  |
|---------|----------|----------|
| 1       | Q-values | Q-values |
| 50      | Q-values | Q-values |
| 100     | Q-values | Q-values |

### (c) Final Learned Policy + Reward Graph

```
# Experiment 2(c): Final Policy + Reward Graph

policy = []
for s in range(16):
    if s == 15: policy.append("G")
    elif s in [5, 10]: policy.append("X")
    else: policy.append(actions[np.argmax(Q[s])][0])

print("Final Learned Policy:")
print(np.array(policy).reshape(4, 4))

# Cumulative reward
cum_reward = np.cumsum(reward_history)
plt.plot(cum_reward)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.grid()
plt.show()
```

## Output

```
Final Learned Policy:  
  
[['R' 'R' 'D' 'D']  
 ['D' 'X' 'D' 'D']  
 ['R' 'R' 'X' 'D']  
 ['R' 'R' 'R' 'G']]
```

## Reward Graph

